GE 1502:
Pur-fect Hydration

13 April 2026

Storm Chasers

Amin Charepoo
Paulpavich Sombunsakdikun
Lucas Azout
Dominic Carr

1. Executive Summary

Through Pur-fect Hydration, we gamified drinking water to help college students who struggle with hydration throughout the day. Our device uses a Raspberry Pi Pico, HX711 load cell, LCD screen, and keypad to measure water consumption and display a cat character that responds to the user's hydration progress. The users must input their weight and activity level to receive a personalized daily water intake goal, and the cat's expression changes from thirsty to happy as they hit certain checkpoints throughout the day. This turns hydration into an engaging, rewarding game.

Our device was created over four design modules. We started from a single-box prototype and finalized with a visually pleasing, compact, two-box laser-cut enclosure that includes 3D-printed components, a keypad, and a buzzer for audio feedback. We had user testing validate our core idea and recommend design improvements, and a later expo presentation with feedback that confirmed strong engagement with the gamification concept, and a successful product. Our final product came in under the $75 budget at $35.10 in costs, met all size and safety constraints, and achieved weighing accuracy to 0.01 oz. Future improvements would include a wireless power source, support for larger water containers, and a smaller, more compact product.

2. Problem Statement + User Persona

Problem Statement

Function

Track and gamify the action of drinking water throughout the day. An engaging hydration device that gives feedback based on your performance.

Objective

The design must have a gamification feature. It must turn the action of drinking water into a game-like activity consistently. Design must be accurate to a certain degree. Be able to consistently track the water level. Must be motivating. See an increase in water consumption compared to not using our design. Rewarding mechanic to the game. So users have something to work towards. Aesthetically appealing design so that it won’t be a pain to look at.

Constrain

A limit of our design is that it can only track one user at a time. Our design doesn't know if two people are using the device. The cost of the entire project must not exceed $75, limited by our budget. The project must fit in a 29"×13"×14" storage space. Electronics must be protected from water splashes. All materials in contact with drinking water must be food-safe and non-toxic

Target user group

18-22-year-old students who don't constantly drink water. / Health conscious individuals who want to keep track of their hydration.

User Persona

Bob Bee is a 19-year-old mechanical engineer at the University of Waterloo that loves spending time outdoors. He is super active, rides his bike, to and around campus, along with playing intramural sports, skiing, hiking, board games, and reading. Bob has a twin sister, and he struggles heavily with OCD. Bob Bee is very active, but unfortunately, doesn't drink enough water. This has resulted in health implications like cramping, dehydration headaches, and many more potential future health concerns.

3. Research Summary

Successful gamification usually includes elements such as instant gratification, clear goals or missions, visible progress tracking, and positive feedback [1] [2]. Having these features increase motivation and engagement by making tasks feel rewarding and purposeful. By applying these principles drinking water can become more engaging and fun.

College students often struggle with hydration due to environmental and behavioral factors. Cafeterias frequently place water dispensers next to soda machines, making sugary drinks easier to get and more enticing [3]. Additionally, unpleasant water taste and the desire to maximize meal plan value also discourages drinking water. Many students also rely on energy and caffeinated drinks for academic and athletic performance, which can replace regular water consumption [4].

Research shows that better hydration can improve school performance. Informational campaigns increase awareness but fail to overcome the appeal of sugary drinks [5]. Making water more visible in cafeterias helps a little but does not address taste or convenience issues. Incorporating motivation strategies like rewards and progress tracking can encourage users to build better hydration habits.

4. Final Prototype Description

Our final prototype is a fully functional hydration game built around an HX711 load cell, Raspberry Pi Pico, LCD screen, and keypad, housed in two laser-cut wooden boxes that separate the Pico wiring from the load cell. A 3D-printed pad on top acts as the weigh station, screwed into the load cell for stability and accuracy.

The device tracks daily water consumption and gamifies it through a cat on the LCD screen, displaying a happy expression when the user meets their hydration goals and a thirsty expression when they fall behind. Users input their weight and activity level through a keypad, and the system calculates a personalized daily water target, awarding points based on the percentage of that goal consumed.

The load cell measures deformation caused by the weight of the water bottle through electrical resistance, and converts that into ounces consumed. Compared to the previous prototype, this version is significantly more compact because of a smaller breadboard and a separate wiring box. The 3D-printed bottle pad replaced a big platform, reducing placement-based errors in measurements. Timed checkpoints were added to track hydration

throughout the full day, and a minimum weight-change threshold was implemented to filter out random measurement errors from slight bottle repositioning.

5. Prototyping Process Summary

Our initial idea focused on hydration with water bottles attachments. We considered a tilt detector, a straw lift tracker, and a manual input button on the side of the bottle, but realized these would not be versatile or functional enough for broader user testing.

For module 2, we pivoted to a desk gadget. We built a smart water bottle tracker using a Raspberry Pi Pico, HX711 load cell, and 0.96" OLED display. The user inputs their weight and activity level, then measures their water bottle before and after drinking. The Pico calculates ounces consumed versus daily goals and rewards progress with a cat display that turns happy when the goal is met. The prototype achieved weighing accuracy to 0.01 oz, though compactness and full-day operation were not yet tested. Feedback focused on aesthetics, versatility, and usability, specifically hiding wires, replacing the laptop keyboard with a keypad, and adding clearer instructions.

In Module 3, we separated the Pico wiring and load cell into a two-box design which improved portability. A 3D-printed holder matching the water bottle bottom improved weighing accuracy and user interaction. We integrated a keypad for user input and added timed hydration checkpoints throughout the day.

User testing was successful with no bugs encountered other than the keypad occasionally missing quick presses. Feedback included displaying recommended water intake after the quiz, supporting more bottle sizes, and adding audio or stronger visual notifications.

For the final design, we focused on aesthetics and expo experience. We painted the enclosure, added more LCD outputs, and adjusted checkpoint thresholds so expo users could reach the happy cat more easily.

6. Expo Presentation

We presented the expo by giving a description of the project and letting people try out the project with either their own water bottle or our water bottles. Users could either drink from their own water bottle and get points or use our full and empty water bottles to test the project. A change that should be made if we were to do another expo demonstration would be to lower the threshold of getting a happy cat since some people have to drink a lot of water in the day and it was hard for them to drink enough water in the short time they had with us.

7. Professional Outcomes

This project taught us how to take an idea that started broad and unfocused and narrow it into something real, functional, and purposeful through continuing to work on it and brainstorm improvements. We learned that the technical side of a project, although it is important, is only as valuable as how well it serves the real people that will actually be using it. Working with the HX711, Raspberry Pi Pico, and keypad taught us that hardware and software have to be

developed together, because an error on one side always creates an error on the other side. Most importantly, we learned that listening to user feedback is not just a step in the process, it is the whole process.

We gained hands-on experience building a fully functional gamified system from scratch, which none of us had done before coming into this course. Presenting at the expo gave us real experience explaining a technical product to an audience that was completely unfamiliar with it, which forced us to think differently about how we communicate our work, and understand it. Going through multiple prototypes gave us firsthand experience with the engineering design process in a way that no lecture or textbook could replicate, without the real experience. We also gained experience working as a team under deadlines, dividing tasks, coming together to make sure every part of the project fit into one coherent final product, and making sure that you understand everything going on even if it is not your particular task.

8. Work Distribution

Team Member	**Individual Tasks**	**Group Tasks**
Amin	Code the Weighing System Code the screen display and animation Code the keypad interaction Solder the wires for the HX711 Research how the HX711 and oled display worked	Wiring the pico to the components Code the milestone and points system Find images for the cat animation
Paulpavich	Idea generation in the early stage. Sketches of possible designs. Physical mechanisms of the project. Securely mounting the load cell. Housing for electronic components. Autocad and slicing of parts from a 3D printer. Designing water bottle housing. How to attach the water bottle to the load cell.	Painting and finishing of the outer surface. 3D printed parts for the group. Assembling the project. Reports, meeting notes, and presentations.
Lucas	User interaction code and increasing the real-world functionality of the project. Researching and implementing the exact quantity of water and all numbers associated with hydration percentage and measurements. Run substantial tests with all the different possible errors to increase versatility once	Laying the foundation for the project structure and functionality goal for each module Writing reports and notes for milestones

	again.	
Dominic	Creating a small enclosure for the pico components and the load cell by lasercutting wood which I designed in AutoCad. 3D printing a case for the screen which shows the animation for the cat. Optimizing space in the design so it could be small enough for a desk. Drawing sketches for prototypes so we could plan the layout of the project.	Assembling the structure of the project and making the project structurally sound. Designing the project so it is visually appealing. Writing reports and meeting notes. Adding the pico component into the physical component of the project.

9. Budget

Item	Unit Value	Units	Qty	Value	Cost	Source	MFR PN/Link	Notes
Raspberry Pi	$5.00	ea	1	$5.00	$0	School	https://www.adafruit.com/pico?src=raspberrypi	Given For Free by School
HX711 Load Cell	$10.61	ea	1	$10.61	$10.61	Amazon	https://tinyurl.com/4acduc7j	2 Pack
Water Bottle	$2.00	ea	2	$4.00	$4	Wollaston	Bought in person	water bottles used for testing
Wood	$12.00	ea	1	$12	$12	Bookstore	Bought in person	Wood for laser cut
0.96 Inch OLED Display	$2.83	ea	3	$8	$8	Amazon	https://www.amazon.com/ELEGOO-Display-Compact-Self-Luminous-Projects/dp/B0D2RMQQHR/ref=sr_1_3?sr=8-3	3-pack was the smallest amount we could get for cheap -> 2 unused
			Totals	$40.1	$35.1			

10. Reflection

Dominic Carr

This project allowed me to get a better understanding of the needs in order to create a project that is for a wide range of people. This project taught me how to work with a real-life example on an actual project of my choice which is used to benefit someone. During this project I became better at modeling using Solidworks and Autocad but also became better at adjusting to mistakes that are made along the way. I modeled a case for a screen which I was able to 3D print and I also modeled the enclosure for our project which was made from lazercut wood. I increased my experience in physical aspects of engineering, but also engaged in adapting my original plan in order to accommodate the user. Over the course of the project our group continuously needed to make adjustments while keeping the users best interests in mind. An

improvement that I believe would take this project to the next level would be to reduce the size of the water tracking device to make it very accessible for the user. I would have also liked to attach a power source to the pico so the device did not need to be connected to the computer. Overall, this project taught me how to model 2D and 3D structures while thinking of user feedback and their needs.

Lucas Azout

This project taught me a substantial amount about how to make a real-world project meant to help people. Regardless of the technical skills I improved upon, like Python and Raspberry Pi Pico kits, being able to adjust and adapt a project to better fit the needs of the people it is meant for was my main priority within this project. I learned to treat every step with purpose and always think about the real person that we were trying to help. One thing I could have improved upon was that I had the same person in mind when thinking about possible users. This resulted in a final product that could only measure a select few sizes of waters, and disregarded my own personal functionality since I carry around a 1 gallon jug of water everywhere I go. I was so focused on helping a specific person that I did not even think about how this project could help myself.

Amin Charepoo

This project taught me a lot about user interface and sensor components. When designing the code, the keypad, and the weighing process I had to constantly think about how would a person that knows nothing about the project use it and what problems could arise. I had to constantly reflect on what a user could do to break the project or what was not as intuitive as I would hope in the project. Using the HX711 taught me about how to use sensor readings and translate it to user-readable data (ounces in this project). Something I would want to improve upon was learning more about user interface to make the interaction as seamless as possible since there were some hiccups with user testing in how intuitive the keypad was and how readable the directions were.

Paulpavich Sombunsakdikun

Throughout the entirety of this project, I felt super fortunate to be able to work with new components and sensors that I've never worked with before. The load cell, key pad, and LCD screen gave me great insight into how to make a project where the user's input matters so much. The challenge of figuring out how to calibrate and set up the load cell was a nice experience to have. I reflected on how I could make the structure more rigid while not restricting the load cell's slight movement. The LCD screen made me consider 3D printing housings for each electrical component. Designing and 3D printing the water bottle holder let me work through the entire process of part design, from concept through manufacturing. Overall, this project gave me valuable experience in making a real-world product that can be useful to people. In the future, I could've made the water bottle housing more universal with larger structures and made the electronic housings more seamless.

APPENDIX

Citations

[1] S. L. Schulte, “Patterns to improve user experience with gamification,” in Proceedings of the 28th European Conference on Pattern Languages of Programs, Irsee Germany: ACM, Jul. 2023, pp. 1–8. doi: 10.1145/3628034.3628065.

[2] M. Cencelj, “A Gamification Project Guide: To a Better Success Experience,” M.I.S., Universidade NOVA de Lisboa (Portugal), Portugal, 2023. Accessed: Jan. 22, 2026. [Online]. Available: https://www.proquest.com/docview/3059434810/abstract/CB6BF6703CD44977PQ/1

[3] A. L. Montuclard, J. Park-Mroch, A. M. J. O’Shea, B. Wansink, J. Irvin, and H. H. Laroche, “College Cafeteria Signage Increases Water Intake but Water Position on the Soda Dispenser Encourages More Soda Consumption,” J. Nutr. Educ. Behav., vol. 49, no. 9, pp. 764-771.e1, Oct. 2017, doi: 10.1016/j.jneb.2017.05.361.

[4] S. Attila and B. Çakir, “Energy-drink consumption in college students and associated factors,” Nutrition, vol. 27, no. 3, pp. 316–322, Mar. 2011, doi: 10.1016/j.nut.2010.02.008.

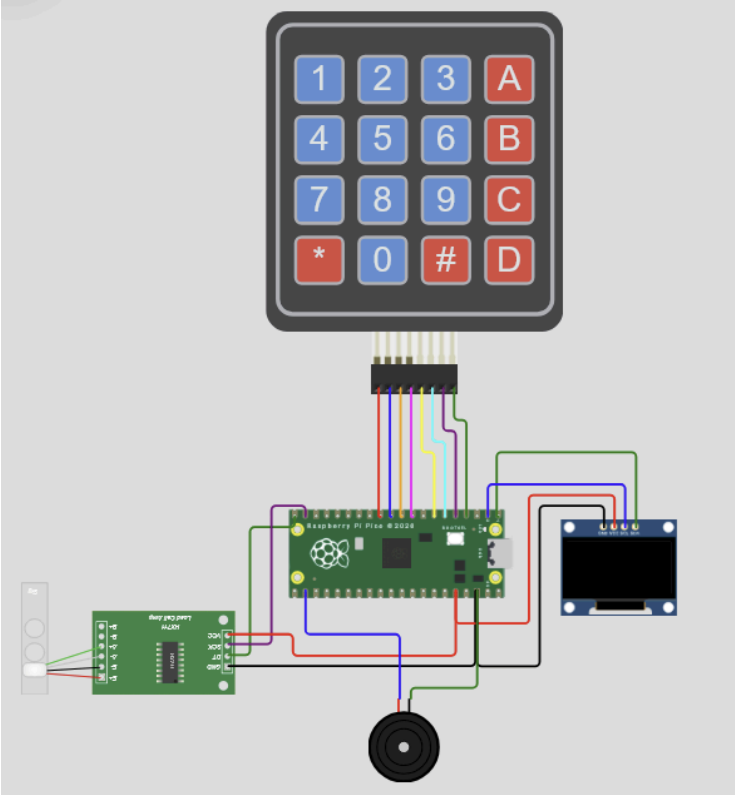
[5] R. Kelly, R. Calabro, L. Beatty, K. Schirmer, and D. Coro, “Evaluating campaign concepts aimed at replacing sugar-sweetened beverages with water,” Health Promot. J. Austr., vol. 36, no. 1, p. e903, 2025, doi: 10.1002/hpja.903.

Appendix 1: Photo Log

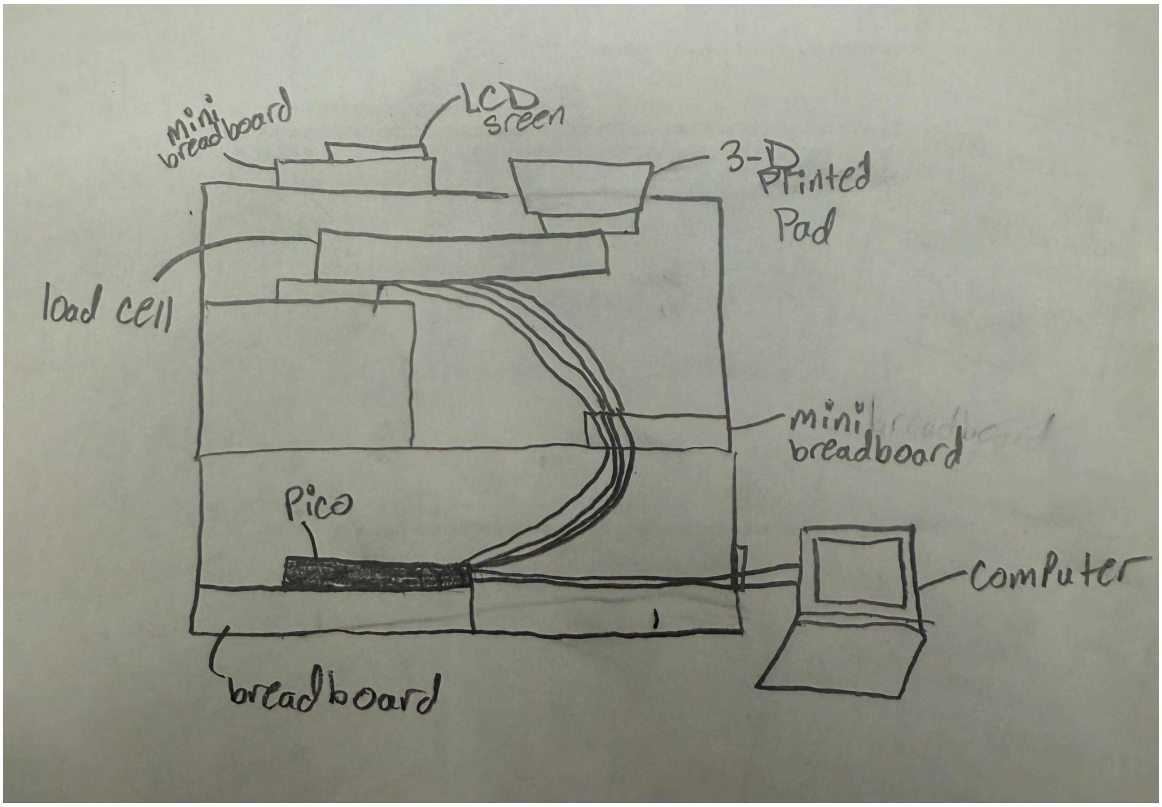
New Screen Case Installed

Painted Project

Expo Setup



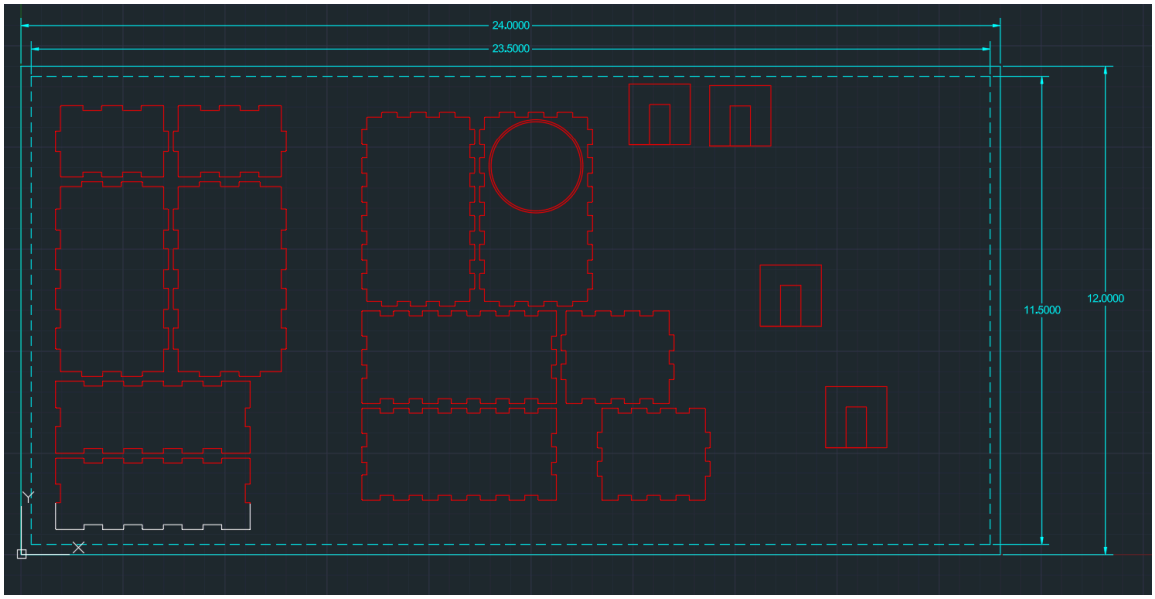
Wiring Diagram



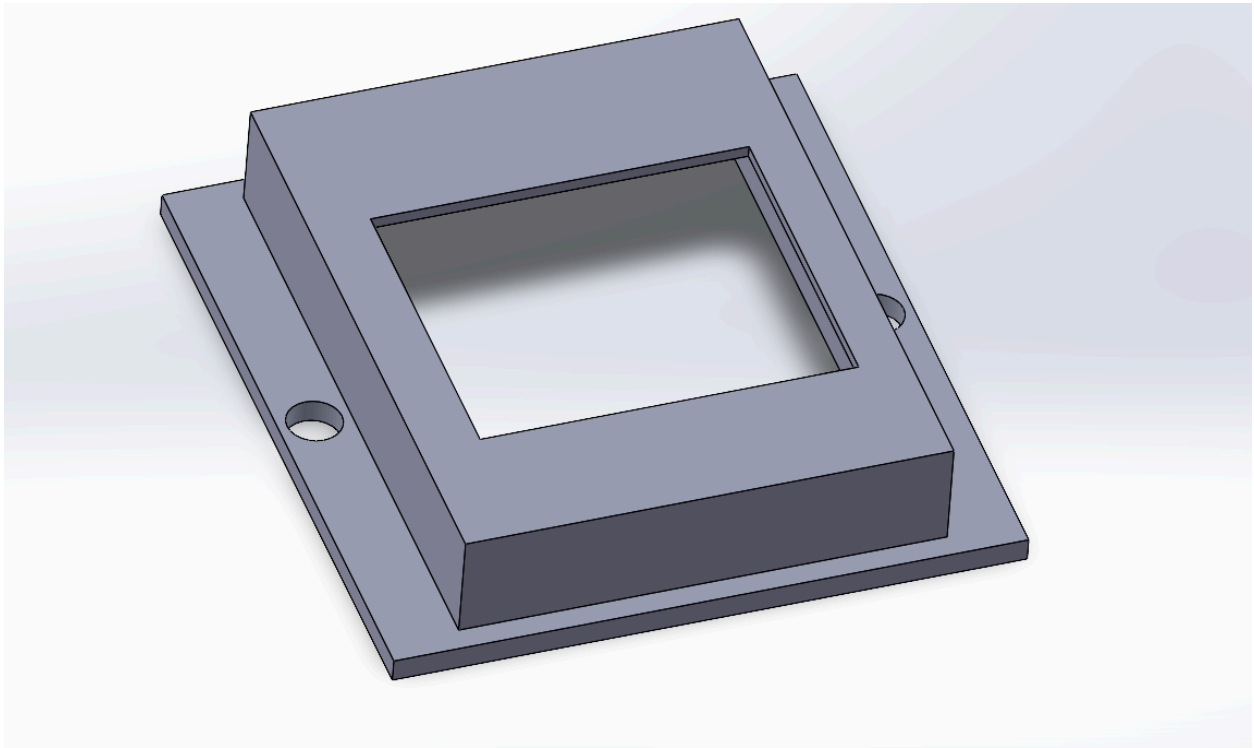
Internals Drawing

Appendix 2: CAD

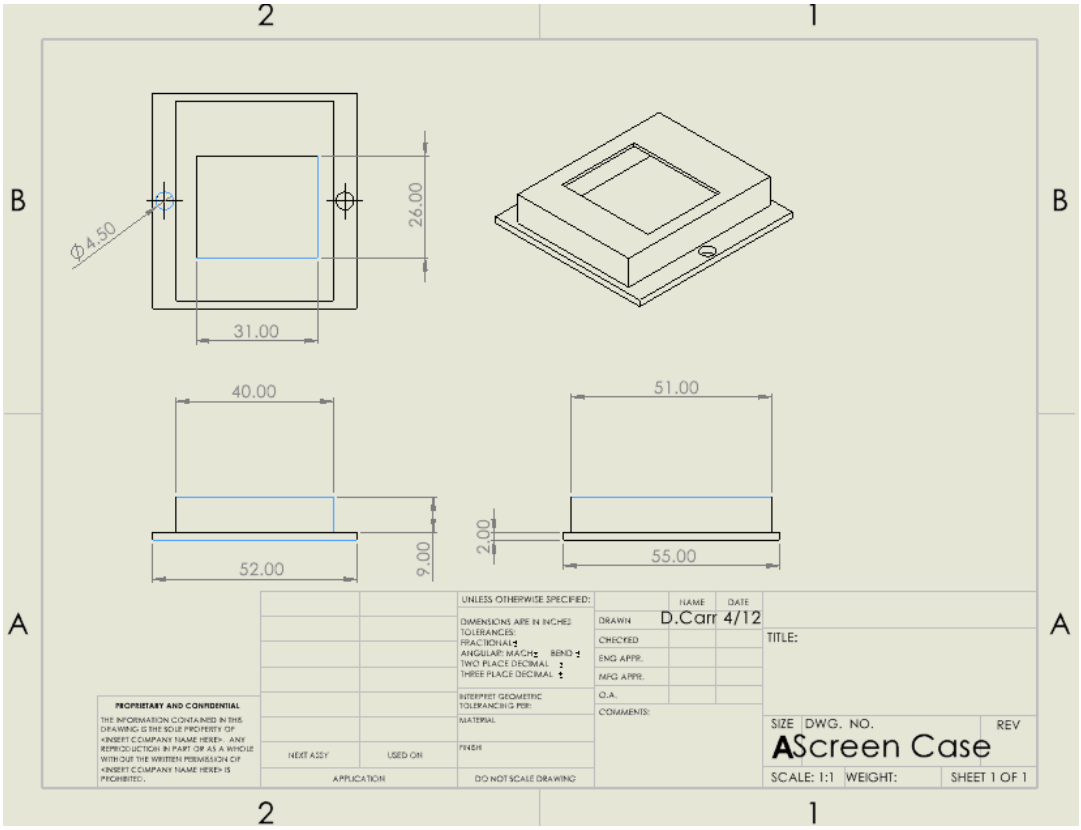
- Include screenshots of any and all CAD files and drawings



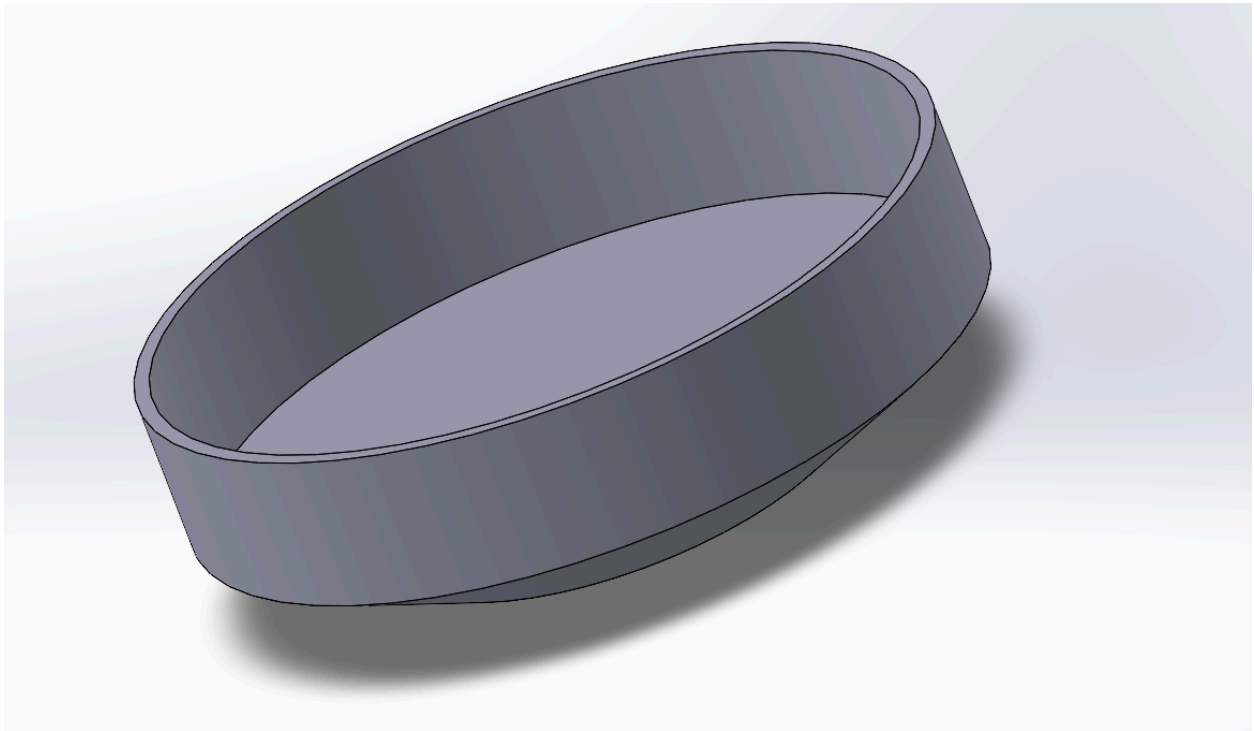
Cad part for structure of design



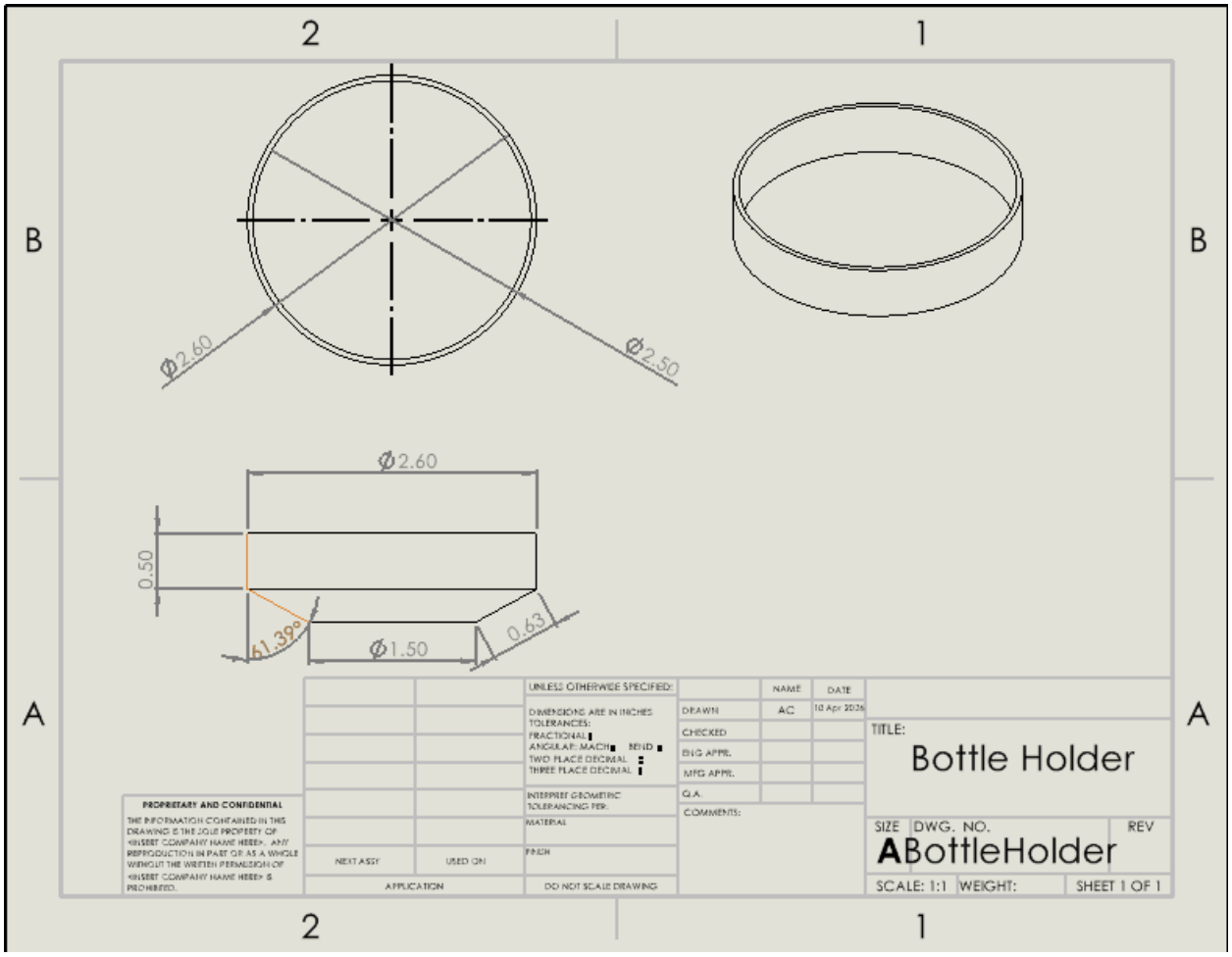
Case for the LCD Screen (Solidworks)



Case for the LCD Screen, Drawing (Solidworks)



Water Bottle Holder (Solidworks)



Water bottle Holder Drawing (Solidworks)

Appendix 3: Code (PICO, Matlab, etc.)

```
"""
Amin Charepoo, Paulpavich Sombunsakdikun, Dominic Carr, Lucas Azout
Professor Jay
Measures the weight of an object using an HX711 and designates points
hx711 repository: https://github.com/robert-hh/hx711
rounding: https://www.w3schools.com/python/ref_func_round.asp
oled display: https://www.tomshardware.com/how-to/oled-display-raspberry-pi-pico
Matrix touchpad:
https://peppe8o.com/use-matrix-keypad-with-raspberry-pi-pico-to-get-user-codes-input/
"""
```

```
from machine import Pin, I2C, PWM
from hx711_gpio import HX711
import time
from ssd1306 import SSD1306_I2C
import framebuf
from imagesOled import catOpen, catClosed, catHappy

# ----------------HX711 pins ------------------------------------
dt = Pin(13, Pin.IN)
sck = Pin(12, Pin.OUT)
hx = HX711(clock=sck, data=dt)

TARE_OFFSET = -482424.76
scale_factor = -13203.0520


# --------------USER SETUP -----------------
playerPoints = 0
prevMeasurement = 0

dailyWaterIntake = 110 # average daily water intake in oz
initialWater = 0
totalDrank = 0

# ------- sleep setup --------
sleepTime = 0.01


# -------- DISPLAY SETUP -----------
WIDTH = 128
HEIGHT = 64
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=200000)
oled = SSD1306_I2C(WIDTH, HEIGHT, i2c)
oled.fill(0)

# -------- IMAGES SETUP -----------
happyCatIMG = framebuf.FrameBuffer(catHappy, 84, 64, framebuf.MONO_HLSB)
closedMouthCatIMG = framebuf.FrameBuffer(catClosed, 64, 64, framebuf.MONO_HLSB)
openMouthCatIMG = framebuf.FrameBuffer(catOpen, 64, 64, framebuf.MONO_HLSB)
imgs = [happyCatIMG, openMouthCatIMG, closedMouthCatIMG]
```

```
imgState = "Thirsty"    #"Happy", "Thirsty"
catState = "Closed" #"Closed", "Open"

lastDisplayedCat = 0    # ms
catDebounce = 10        # ms
happyThreshold = 25     # percent
lastHappyPercent = 0.0  # percent


# ---- TIMER SETUP ----
thirst_timer = 10 * 1000  # ms (10ms * 1000ms/s = 10s)
lastMilestone = time.ticks_ms()

# ------- KEYPAD SETUP --------
col_list = [2, 3, 4, 5]
row_list = [6, 7, 8, 9]

for x in range(4):
    row_list[x] = Pin(row_list[x], Pin.OUT) # setup each row pin as output
    row_list[x].value(1)                    # set each one to high

for x in range(4):
    col_list[x] = Pin(col_list[x], Pin.IN, Pin.PULL_UP) # setup each col pin as input

key_map = [
    ["D", "#", "0", "*"],
    ["C", "9", "8", "7"],
    ["B", "6", "5", "4"],
    ["A", "3", "2", "1"],
]

# ------ DISPLAY FUNCTIONS ---------
def displayImg(index, x, y):
    oled.fill(0)
    oled.blit(imgs[index], x, y)
    oled.show()

def displayText(msg):
    oled.fill(0)
```

```
    subMsg = ""
    yPos = 0
    xPos = 0
    for i in range(len(msg)):
        c = msg[i]                  # get current character
        if c != " ":                # if the character is not a space
            subMsg += c             # add character to current word
            if xPos + len(subMsg) > 16: # if the word is too big to fit on line
                yPos += 1 #go to next row
                xPos = 0 # reset x coordinate
            if i == len(msg) - 1:
                oled.text(subMsg, 8*xPos, 8*yPos) # put the message at correct coordinate
        else: # current character is a space
            oled.text(subMsg, 8*(xPos), 8*yPos) # put the message at correct coordinate
            xPos += len(subMsg)             # shift up x position
            if xPos == 16:                  # if it is at the end of the row don't print a space
                xPos = 0                    # reset xPos
                yPos += 1                   # shift down a row
                subMsg = ""
            else: # it is not at end of row so print a space
                oled.text(" ", 8*xPos, 8*yPos)
                xPos += 1     # shift up on position
                subMsg = ""   # reset word
    oled.show()

# --- KEYPAD FUNCTIONS ---
def Keypad4x4Read():
    for r_idx, r in enumerate(row_list):   # pull ONE row low at a time
        r.value(0)
        time.sleep_us(10)              # let signal settle
        for c_idx, c in enumerate(col_list):
            if c.value() == 0:         # key pressed in this row
                r.value(1)
                return key_map[r_idx][c_idx]
        r.value(1)                     # restore before next row
    return None

def waitForRelease(): # protects against double pressing
    while Keypad4x4Read() is not None:
        time.sleep_ms(10)
```

```
    time.sleep_ms(50)  # extra debounce after release

def waitForKey(key = "*"):
    while Keypad4x4Read() != key:
        continue
    waitForRelease()

# ---- SCALE FUNCTIONS ----
# If the offset and scale factor are consistent use this so we don't have to calibrate every time
def setupScale():
    hx.OFFSET = TARE_OFFSET
    hx.set_scale(scale_factor)
    return 0

def read_average(times=3, delay=0.05):
    total = 0
    for i in range(times):
        total += hx.get_units()
        time.sleep(delay)
    return total / times

# ---- CALIBRATION ----
def calibrateWeight():
    print("Warming up...")
    displayText("Warming up...")

    # Tare
    print("Place nothing on the scale and press * to tare")
    displayText("Place nothing on the scale and press * to zero scale")
    waitForKey("*")
    hx.tare() # ZERO THE SCALE
    print("Tare done, offset: ", hx.OFFSET)

    # User provides known weight
    print("Place your full waterbottle on the scale and press * to measure enter its known weight
in oz: ")
    displayText("Place bottle on scale. Enter weight in oz: ")
    user_input = 0
    key = ""
    while key != "*":
```

```
        key = Keypad4x4Read()
        if key != None and key != "*":
            print("You pressed: " + key)
            if key.isdigit():
                user_input = user_input * 10 + int(key)
                displayText("Place bottle on scale. Enter weight in oz: " + str(user_input))
                print("Input so far:", user_input)
            else:
                print("Invalid key")
            waitForRelease()
        elif key == "*":
            waitForRelease()

    print("user_input:", user_input)
    if user_input == 0:
        known_weight = 17
    else:
        known_weight = user_input
    print("known_weight", known_weight)

    # Take raw reading
    print("Reading raw value...")
    time.sleep(1)  # wait a bit for stabilization
    raw_value = hx.read_average()  # tare-corrected raw value
    print("Raw value measured:", raw_value)

    # Calculate scale factor
    scale_factor = (raw_value - hx.OFFSET) / known_weight # raw value per unit of weight
    hx.set_scale(scale_factor)
    print("Calibration complete. Scale factor:", scale_factor)
    displayText("Complete")

    return known_weight



# ---- IMAGE FUNCTIONS ----
def checkImageState():
    global catState, lastDisplayedCat, imgState, lastNotified
    if imgState == "Happy":
```

```
        displayImg(0, 22, 0) # img[0] = Happy, x = 32, y = 0
    elif imgState == "Thirsty":
        if (time.ticks_ms() - lastNotified > notificationBuffer):
            playSong(notificationSound)
            lastNotified = time.ticks_ms()
        if (time.ticks_ms() - lastDisplayedCat < catDebounce):
            return
        elif catState == "Closed":
            displayImg(1, 48, 0) # img[1] = Open, x = 0, y = 0
            lastDisplayedCat = time.ticks_ms()
            catState = "Open"
        elif catState == "Open":
            displayImg(2, 48, 0) # img[1] = Closed, x = 0, y = 0
            lastDisplayedCat = time.ticks_ms()
            catState = "Closed"
        else:
            print("INVALID CAT STATE")
            catState = "Closed"
            checkImageState()
    else:
        print("INVALID STATE")
        imgState = "Happy"
        checkImageState()

def checkThirst() :
    global imgState
    if imgState == "Happy":
        elapsed = time.ticks_ms() - lastMilestone
        if elapsed >= thirst_timer:
            imgState = "Thirsty"
            print("Cat is thirsty, did not reach milestone in time")

# --- WATER QUIZ ----
def waterQuiz():
    print("How much do you weight in pounds? (press * to submit) ")
    displayText("How much do you weigh in pounds? (press * to submit)")
    key = ""
    weight = 0
    while key != "*":
        key = Keypad4x4Read()
```

```
        if key != None and key != "*":
            print("You pressed: " + key)
            if key.isdigit():
                weight = weight * 10 + int(key)
                displayText("How much do you weigh in pounds? (press * to submit): " + str(weight))
            else:
                print("Invalid key")
            waitForRelease()
        elif key == "*":
            waitForRelease()
    if (weight == 0):
        print("Invalid weight, defaulting to 175")
        weight = 175
    print("weight: ", weight)

    print("How active are you on a scale from 1-5? ")
    displayText("How active are you on a scale from 1-5? ")

    key = ""
    activity = 1
    while key != "*":
        key = Keypad4x4Read()
        if key != None and key != "*":
            print("You pressed: " + key)
            if key.isdigit() and int(key) in range(1, 6):
                activity = int(key)
                displayText("How active are you on a scale from 1-5?: " + str(activity))
            else:
                print("Invalid key")
            waitForRelease()
    waitForRelease()
    print("activity level: ", activity)
    water_oz = weight * 0.5
    water_oz *= (1 + 0.1 * (activity - 1))
    displayText(str(water_oz))
    time.sleep(2)
    return water_oz #this is how much water the person should drink per day

# ---- POINTS AND HYDRATION ----
def calculatePoints(weight, playerPoints, prevMeasurement, dailyWaterIntake):
```

```
    global totalDrank
    if weight > prevMeasurement: # check if weight has increased
        return playerPoints, weight

    difference = prevMeasurement - weight
    totalDrank = totalDrank + difference
    print("Diff:", difference)

    if initialWater == 0:
        print("Initial water is 0")
        return playerPoints, prevMeasurement

    if 100 * difference/initialWater < 5: # check if difference is too small for a point increase
        return playerPoints, prevMeasurement

    points = 100 * difference/dailyWaterIntake

    hydrationPrint()

    return playerPoints + points, weight


def hydrationPrint():
    print("""  / \__
     (    @\___
     /         O
    /   (_____/
     /_____/   U
    """)
    print("Your pet is ", round((totalDrank/dailyWaterIntake)*100, 2), " % hydrated!")


def pointsToImage():
    global lastHappyPercent, lastMilestone

    # current hydration percent
    if(dailyWaterIntake > 0):
        currentPercent = (totalDrank / dailyWaterIntake) * 100
    else:
        currentPercent = 0
```

```
    # trigger happy only when you cross the next +15% milestone
    if currentPercent >= lastHappyPercent + happyThreshold:
        lastHappyPercent += happyThreshold
        lastMilestone = time.ticks_ms()
        print("Milestone reached!")
        return "Happy"
    else:
        return None

def check_checkpoints(reached_25, reached_50, reached_75, reached_100):
    percent = playerPoints
    if percent >= 1 and not reached_25:
        reached_25 = True
        print("25% reached!")
        displayImg(0, 22, 0)
        playSong(completedMilestoneSound)
    if percent >= 50 and not reached_50:
        reached_50 = True
        print("50% reached!")
        displayImg(0, 22, 0)
        playSong(completedMilestoneSound)
    if percent >= 75 and not reached_75:
        reached_75 = True
        print("75% reached!")
        displayImg(0, 22, 0)
        playSong(completedMilestoneSound)
    if percent >= 100 and not reached_100:
        reached_100 = True
        print("100% reached!")
        displayImg(0, 22, 0)
        playSong(completedMilestoneSound)
    return reached_25, reached_50, reached_75, reached_100

# ---- BUZZER SETUP ----

buzzerPin = 17
buzzer = PWM(Pin(buzzerPin))
```

```
# frequency for notes
C5 = 523
E5 = 659
G5 = 784
C6 = 1047

lastNotified = 0 # ms
notificationBuffer = 10*1000 # ms
playedVictory = False # track if already played completedMilestoneSound

# sound effects
notificationSound = [
    {"beats": .5, "freq": C5},
    {"beats": .5, "freq": E5},
]

completedMilestoneSound = [
    {"beats": .5, "freq": E5},
    {"beats": .5, "freq": C5},
    {"beats": .5, "freq": E5},
    {"beats": .5, "freq": G5},
    {"beats": .5, "freq": E5},
    {"beats": .5, "freq": G5},
    {"beats": 2, "freq": C6},
]

# buzzer functions
def playtone(frequency):
    buzzer.duty_u16(1000)
    buzzer.freq(frequency)

def bequiet():
    buzzer.duty_u16(0)

def playBeats(note):
    playtone(note["freq"])
    time.sleep(60/150 * note["beats"])

    bequiet()
```

```
def playSong(mySong):
    for note in mySong:
        playBeats(note)
    bequiet()


# ---- STARTUP ----
setupScale()

reached_25 = False
reached_50 = False
reached_75 = False
reached_100 = False

print("PRESS * TO START")
displayText("PRESS * TO START")
waitForKey("*")

print("Place bottle on scale and press * to measure new bottle weight ")
displayText("Place bottle on scale and press *")
waitForKey("*")

prevMeasurement = read_average()
if (prevMeasurement < 0):
    prevMeasurement = 0
initialWater = prevMeasurement
print("Initial water:", initialWater)

"""
prevMeasurement = calibrateWeight()
initialWater = prevMeasurement
"""
dailyWaterIntake = waterQuiz()  # oz
print("Daily water intake: ", dailyWaterIntake, "oz")




# --- MAIN LOOP ----
while True:
```

```
    checkThirst()
    checkImageState() # update image on screen
    key = ""
    while key != "*":
        key = Keypad4x4Read()
        checkThirst()
        checkImageState()
        weight = read_average()
        #print("Weight:", weight)
        time.sleep(sleepTime/2)
    waitForRelease()

    # measure new water bottle weight
    print("Place bottle on scale and press * to measure new bottle weight ")
    displayText("Place bottle on scale and press *")
    waitForKey("*")

    weight = read_average()
    if (weight < 0):
        weight = 0

    if prevMeasurement == 0:
        prevMeasurement = weight
        initialWater = prevMeasurement
        print("Initial weight:", initialWater, "oz")
    else:
        print("Previous weight:", prevMeasurement, "oz, Measured Weight:", weight, "oz")
        playerPoints, prevMeasurement = calculatePoints(weight, playerPoints, prevMeasurement,
dailyWaterIntake)
        reached_25, reached_50, reached_75, reached_100 = check_checkpoints(reached_25,
reached_50, reached_75, reached_100)
        print("Points:", playerPoints)
        displayText("Points: " + str(playerPoints))
        time.sleep(3)


    newImgState = pointsToImage() # check if milestone got reached
    if newImgState is not None: # only update if milestone is crossed
        imgState = newImgState
```

```
time.sleep(sleepTime)
```

Appendix 4 : Final Team Meeting Notes

- 02/26/26
    - Create plan for milestone 3
    - Brainstorm additions
    - Set goals for over spring break
    - Add functionality to code
    - Schedule for Fyleic to build
- 03/10/26
    - Go over everything done during break
    - Set checklist to complete for milestone 3
    - Start building prototype
    - Designing 3d printed parts
    - Create more effective wiring for pico
    - Tried to buy wood, ended up with birch
    - Add matrix keypad/brainstorm how we will incorporate the keypad
- 03/12/26
    - Discuss all pros and cons of current project
    - Lay out what we need to get done next week
    - Use class time effectively to plan and organize
- 03/17/26
    - Build in Fyelic
    - Finish 3d printing
    - Finish Lasercutting
    - Get all parts ready and begin assembling
    - More parts are 3d printed to be picked up later
- 03/19/26
    - Fix up code
    - Fix up wiring
    - Bring everyone together on what we did, and what we will do
    - Communicate finishing module 3
    - Brainstorm nice to haves
    - Add the pico component into the physical prototype
- 03/24/26
    - ASSEMBLE
    - FINISH ASSEMBLING AND GET READY FOR MODULE 3 SUBMISSION
    - Print out persona and qr code for feedback

- Test timing code to see if milestones are reached and reset after a certain amount of time
- Test consistency of weight readings